

OSSP Practical 12

Practical 12 - Theory

Deadlock Conditions, Detection, Prevention and Avoidance

A **deadlock** is a situation in which a group of processes are permanently blocked because each process is waiting for a resource held by another process.

Necessary Conditions for Deadlock

Deadlock can occur when the following four conditions exist simultaneously:

1. **Mutual Exclusion** – A resource can be used by only one process at a time.
2. **Hold and Wait** – A process holds one or more resources while waiting for additional resources.
3. **No Preemption** – A resource cannot be forcibly taken from a process.
4. **Circular Wait** – Processes form a circular chain where each process waits for a resource held by the next process.

Deadlock Prevention

Deadlock prevention works by ensuring that at least one of the four necessary conditions cannot occur.

Deadlock Detection

Deadlock detection allows the system to operate normally and periodically checks whether a deadlock has occurred. A resource-allocation graph or detection algorithm can be used to identify deadlocks.

Deadlock Avoidance

Deadlock avoidance checks resource requests before granting them and ensures that the system remains in a **safe state**.

Banker's Algorithm

The **Banker's Algorithm** is a deadlock-avoidance algorithm. It calculates whether granting a resource request can leave the system in a safe state.

Important matrices/values are:

- **Allocation** – Resources currently allocated to each process.

- **Maximum** – Maximum resources each process may require.
- **Need** – Remaining resources required by each process.

The Need matrix is calculated as:

Need = Maximum - Allocation

If all processes can complete in some order, the system is in a **safe state**. If no complete safe sequence can be found, the state is considered **unsafe**.

Conclusion

Deadlock occurs when processes wait indefinitely for resources. Deadlock can be handled using **prevention, detection, and avoidance** techniques. In this practical, the **Banker's Algorithm** was used to determine whether the system is in a safe or unsafe state.

```
alurusaranya@LAPTOP-0PA61QEN:~$ mkdir practical12
alurusaranya@LAPTOP-0PA61QEN:~$ cd practical12
alurusaranya@LAPTOP-0PA61QEN:~/practical12$ pwd
/home/alurusaranya/practical12
alurusaranya@LAPTOP-0PA61QEN:~/practical12$ nano banker.c
alurusaranya@LAPTOP-0PA61QEN:~/practical12$ ls
banker.c
alurusaranya@LAPTOP-0PA61QEN:~/practical12$ gcc banker.c -o banker
alurusaranya@LAPTOP-0PA61QEN:~/practical12$ ./banker
Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
7 Enter Maximum Matrix:
5 3
3 2 2
9 0 2
2 2 2
4 3 3
3 3 Enter Available Resources:
2

System is in a SAFE state.
Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2
alurusaranya@LAPTOP-0PA61QEN:~/practical12$ ./banker
Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
```

2520080012

```
Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2
alurusaranya@LAPTOP-0PA61QEN:~/practical12$ ./banker
Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Maximum Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources:
0 0 0

System is in an UNSAFE state.
alurusaranya@LAPTOP-0PA61QEN:~/practical12$ ./banker
Enter number of processes: 2
Enter number of resources: 2
Enter Allocation Matrix:
1 1
0 1
Enter Maximum Matrix:
2 2
2 2
Enter Available Resources:
3 3

System is in a SAFE state.
Safe Sequence: P0 -> P1
alurusaranya@LAPTOP-0PA61QEN:~/practical12$ ls -l

alurusaranya@LAPTOP-0PA61QEN:~/practical12$ ls -l
total 20
-rwxrwxr-x 1 alurusaranya alurusaranya 16152 Sep 28 05:32 banker
-rw-rw-r-- 1 alurusaranya alurusaranya 2360 Sep 28 05:31 banker.c
alurusaranya@LAPTOP-0PA61QEN:~/practical12$ valgrind --leak-check=full ./banker
==927== Memcheck, a memory error detector
==927== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==927== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==927== Command: ./banker
==927==
Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Maximum Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources:
3 3 2

System is in a SAFE state.
Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2
==927==
==927== HEAP SUMMARY:
==927==   in use at exit: 0 bytes in 0 blocks
==927==   total heap usage: 2 allocs, 2 frees, 2,048 bytes allocated
==927==
==927== All heap blocks were freed -- no leaks are possible
```

2520080012

```
0 0 2
Enter Maximum Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources:
3 3 2

System is in a SAFE state.
Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2
==927==
==927== HEAP SUMMARY:
==927==   in use at exit: 0 bytes in 0 blocks
==927==   total heap usage: 2 allocs, 2 frees, 2,048 bytes allocated
==927==
==927== All heap blocks were freed -- no leaks are possible
==927==
==927== For lists of detected and suppressed errors, rerun with: -s
==927== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
alurusaranya@LAPTOP-0PA61QEN:~/practical12$ |
```

```
GNU nano 1.2 dankeer.c
#include <stdio.h>

int main()
{
    int n, m;
    int allocation[10][10], max[10][10], need[10][10];
    int available[10], work[10], finish[10];
    int safeSequence[10];

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resources: ");
    scanf("%d", &m);

    printf("Enter Allocation Matrix:\n");

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < m; j++)
        {
            scanf("%d", &allocation[i][j]);
        }
    }

    printf("Enter Maximum Matrix:\n");

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < m; j++)
        {
            scanf("%d", &max[i][j]);
        }
    }

    printf("Enter Available Resources:\n");
```

```

GNU nano 7.2 banker.c
}

printf("Enter Available Resources:\n");

for (int j = 0; j < m; j++)
{
    scanf("%d", &available[j]);
}

/* Calculate Need Matrix */

for (int i = 0; i < n; i++)
{
    for (int j = 0; j < m; j++)
    {
        need[i][j] = max[i][j] - allocation[i][j];
    }
}

/* Initialize Work and Finish */

for (int j = 0; j < m; j++)
    work[j] = available[j];

for (int i = 0; i < n; i++)
    finish[i] = 0;

int count = 0;

while (count < n)
{
    int found = 0;

    for (int i = 0; i < n; i++)
    {
        if (finish[i] == 0)

GNU nano 7.2 banker.c
        for (int i = 0; i < n; i++)
        {
            if (finish[i] == 0)
            {
                int possible = 1;

                for (int j = 0; j < m; j++)
                {
                    if (need[i][j] > work[j])
                    {
                        possible = 0;
                        break;
                    }
                }

                if (possible)
                {
                    for (int j = 0; j < m; j++)
                        work[j] += allocation[i][j];

                    safeSequence[count] = i;
                    count++;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }

        if (found == 0)
            break;
    }

    if (count == n)
    {
        printf("\nSystem is in a SAFE state.\n");
    }
}

```

2520080012

```
GNU nano 7.2                                     banker.c
        safeSequence[count] = i;
        count++;
        finish[i] = 1;
        found = 1;
    }
}

if (found == 0)
    break;
}

if (count == n)
{
    printf("\nSystem is in a SAFE state.\n");
    printf("Safe Sequence: ");
    for (int i = 0; i < n; i++)
    {
        printf("P%d", safeSequence[i]);

        if (i != n - 1)
            printf(" -> ");
    }

    printf("\n");
}
else
{
    printf("\nSystem is in an UNSAFE state.\n");
}

return 0;
}
```